

Assignment: Image Manipulation Software in C

1. Introduction

In this assignment, you will develop a graphical **Image Manipulation Software** using the C programming language and the IUP (Portable User Interface) toolkit. The objective is to apply structured programming concepts such as functions, structures, arrays, pointers, dynamic memory allocation, and modular programming to develop a practical software application. The application will allow users to open an image, display it, perform different image manipulation operations, and save the modified image. IUP will be used to develop the graphical user interface, while the image manipulation algorithms must be implemented by the students in C.

Students may use an external library for reading and writing the specified image format. However, the actual image manipulation algorithms must be implemented by the students. Students should not simply call ready-made functions that directly perform grayscale conversion, blur, or other required operations. The purpose of using an image library is to avoid requiring students to implement complex image file formats; the main objective of the assignment is to manipulate pixel data using C.

2. Introduction to IUP

IUP is a C-based graphical user interface toolkit that provides components such as windows, buttons, menus, labels, text fields, dialogs, and other interface elements. It is suitable for this assignment because students can develop a graphical application while continuing to use structured programming in C.

An IUP application generally creates GUI components, arranges them into a layout, associates callback functions with user actions, displays the application window, and starts the IUP event loop. The main IUP object is an `Ihandle`, which represents a GUI component. Students should learn how to create windows, buttons, menus, dialogs, and layouts, configure component attributes, and associate callback functions with user actions.

For example, a button can be created using `IupButton()`, multiple components can be arranged using `IupHbox()` and `IupVbox()`, component properties can be configured using `IupSetAttribute()`, and user actions can be handled using `IupSetCallback()`. A callback function should call the appropriate image-processing function. For example, clicking a Grayscale button should invoke a function such as `apply_grayscale()` rather than placing the complete grayscale algorithm inside the GUI callback.

3. IUP Installation

Install the IUP development package appropriate for your operating system and C compiler. The installation should provide the IUP header files and library files required by the compiler and linker. Before starting the project, create a small IUP test program that initializes IUP, creates a simple window and button, displays the window, starts the IUP event loop, and closes IUP when the application terminates. Successfully running this test program confirms that IUP has been configured correctly. The exact installation and compilation procedure depends on the operating system and compiler being used.

4. Project Description

The objective of the project is to develop a small graphical image editor. The application should allow a user to open a predefined image format, view the image, perform different image manipulation operations, undo changes where applicable, and save the resulting image.

Students are free to design the graphical interface, but it should be organized and easy to use. The interface may contain menus, buttons, dialogs, and an image display area for interacting with the different operations. The application should provide clear access to the available image manipulation features.

5. Image Format

To keep the project focused on image manipulation rather than image-format compatibility, all students will work with **24-bit uncompressed BMP (Bitmap) images**. The input and output images should use the `.bmp` format. Students should design their software to open and save only the specified BMP format. The application does not need to support PNG, JPEG, GIF, or other image formats. If the user attempts to open an unsupported file, the application should display an appropriate error message rather than crashing.

6. Image Representation

An image can be represented as a collection of pixels. A pixel can be represented using a structure containing red, green, and blue components. For example, a suitable pixel structure may contain three `unsigned char` values representing the red, green, and blue channels. The complete image can be represented using another structure containing the image width, image height, and a dynamically allocated array of pixels. Students should use this representation to practice structures, arrays, pointers, and dynamic memory allocation. A pixel at coordinate `(x, y)` can be accessed using an expression such as `image->data[y * image->width + x]`.

7. Required Features

The application should provide the following features.

Open Image: The application should allow the user to select and open a 24-bit uncompressed BMP image using an IUP file dialog. The selected image should be loaded into memory and displayed in the application.

Display Image: The currently loaded image should be displayed clearly within the application window. The display should be refreshed whenever an image manipulation operation changes the image.

Save Image: The application should allow the user to save the current image as a BMP file using an IUP file dialog.

Grayscale: The application should convert the color image into grayscale. For each pixel, the program should calculate an intensity value using a weighted combination of the red, green, and blue components. A commonly used formula is `gray = 0.299R + 0.587G + 0.114B`. The resulting value should be assigned to all three color channels.

Brightness Adjustment: The application should allow the user to increase or decrease image brightness. A user-specified value can be added to each RGB component. Values must be restricted to the valid range of 0 to 255.

Image Inversion: The application should produce a negative-like image by calculating `R = 255 - R`, `G = 255 - G`, and `B = 255 - B` for every pixel.

Horizontal Flip: The application should mirror the image from left to right. A pixel at `(x, y)` should be exchanged with the pixel at `(width - 1 - x, y)`.

Vertical Flip: The application should mirror the image from top to bottom. A pixel at `(x, y)` should be exchanged with the pixel at `(x, height - 1 - y)`.

Rotate 90 Degrees: The application should rotate the image 90 degrees clockwise. Since rotation changes the image dimensions, students should create a new image with the width and height exchanged and copy the pixels to their new positions.

Crop: The application should allow the user to extract a rectangular region of the image. The selected region should be converted into a new image. The program should ensure that the selected region remains within the image boundaries.

Blur: The application should provide a blur operation using a 3×3 neighborhood. Each output pixel should be calculated by averaging the corresponding RGB components of the current pixel and its neighboring pixels. Students should use a separate output image so that newly calculated pixels do not affect subsequent calculations.

Undo: The application should provide at least one level of Undo. Before applying an image manipulation operation, the program should preserve a copy of the previous image so that it can be restored when the user selects Undo.

8. Optional Advanced Feature: Image Sharpening

Students may implement image sharpening using convolution as an optional advanced feature. A simple sharpening kernel is:

$$\begin{matrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{matrix}$$

For each pixel, the program should consider the surrounding 3×3 neighborhood, multiply each neighboring pixel value by the corresponding kernel value, and add the results to obtain the new pixel value. The resulting RGB values must be restricted to the range 0 to 255. Students should handle image boundaries appropriately and use a separate output image when necessary. The sharpening algorithm must be implemented by the student rather than using a ready-made sharpening function from an external image-processing library. Successful implementation of this feature can earn bonus marks.

9. Suggested Program Organization

The application should be divided into multiple functions rather than placing the complete program inside `main()`. Students are encouraged to separate the graphical interface, image management, and image-processing operations into different source and header files. For example, `main.c` can contain program initialization, GUI-related files can contain IUP components and callback functions, image-related files can contain image loading, saving, copying, allocation, and deallocation functions, and image-processing files can contain the manipulation algorithms. The GUI callback functions should invoke the appropriate image-processing functions. For example, the callback associated with the Grayscale button should call a grayscale function rather than containing the complete grayscale algorithm itself.

10. Error Handling and Memory Management

The application should handle common invalid situations without crashing. For example, the program should display an appropriate message if the user attempts to manipulate an image when no image has been loaded, attempts to open an unsupported file, enters an invalid brightness value, or specifies a crop region outside the image boundaries.

Students must correctly manage dynamically allocated memory. Memory allocated for image data should be released when it is no longer required. Operations such as rotation, cropping, blur, and undo may require additional image buffers, and these buffers must also be properly managed.

11. Project Demonstration and Viva

During the project demonstration, students should open an image and demonstrate the implemented image manipulation features. Students should be able to explain how the image is represented in memory, how individual pixels are accessed, how their image-processing algorithms work, and how dynamically allocated memory is managed.

During the viva, students may be asked to explain a particular function, trace the execution of an algorithm, identify the purpose of a pointer or structure, explain how an IUP callback works, or make a small modification to their program. Students should demonstrate a clear understanding of their own implementation.

12. Marking Rubric

The project will be evaluated out of **100 marks**. The graphical user interface and use of IUP will carry **10 marks**. Image loading, display, and saving will carry **10 marks**. Implementation of the required image manipulation features will carry **30 marks**. Structured programming and modularity will carry **10 marks**. Dynamic memory management and error handling will carry **10 marks**. Project demonstration , viva and technical understanding will carry **30 marks**.

The optional image sharpening feature may receive **up to 5 bonus marks** when correctly implemented using convolution. The bonus will be awarded only when the convolution algorithm is implemented by the student rather than through a ready-made image-processing function.